Trace2Skill: Distill Trajectory-Local Lessons into Transferable Agent Skills

Jingwei Ni[§,*2,3], **Yihao Liu**[§,*4], **Xinpeng Liu**[§,*4], **Yutao Sun**[§,*5], **Mengyu Zhou**[†1], **Pengyu Cheng**[1], **Dexin Wang**[1], **Erchao Zhao**[1], **Xiaoxi Jiang**[1] **and Guanjun Jiang**[1]

[1]Qwen Large Model Application Team, Alibaba, [2]ETH Zürich, [3]University of Zurich, [4]Peking University, [5]Zhejiang University

[*]Work done during an internship at Alibaba. [†]Corresponding author. [§]Core Contributors.

arXiv:2603.25158v2 [cs.AI] 29 Mar 2026

Equipping Large Language Model (LLM) agents with domain-specific skills is critical for tackling complex tasks. Yet, manual authoring creates a severe scalability bottleneck. Conversely, automated skill generation often yields fragile or fragmented results because it either relies on shallow parametric knowledge or sequentially overfits to non-generalizable trajectory-local lessons. To overcome this, we introduce Trace2Skill, a framework that mirrors how human experts author skills: by holistically analyzing broad execution experience before distilling it into a single, comprehensive guide. Instead of reacting sequentially to individual trajectories, Trace2Skill dispatches a parallel fleet of sub-agents to analyze a diverse pool of executions. It extracts trajectory-specific lessons and hierarchically consolidates them into a unified, conflict-free skill directory via inductive reasoning. Trace2Skill supports both deepening existing human-written skills and creating new ones from scratch. Experiments in challenging domains, such as spreadsheet, VisionQA and math reasoning, show that Trace2Skill significantly improves upon strong baselines, including Anthropic's official xlsx skills. Crucially, this trajectory-grounded evolution does not merely memorize task instances or model-specific quirks: evolved skills transfer across LLM scales and generalize to OOD settings. For example, skills evolved by Qwen3.5-35B on its own trajectories improved a Qwen3.5-122B agent by up to 57.65 absolute percentage points on WikiTableQuestions. Further analysis confirms that our holistic, parallel consolidation outperforms both online sequential editing and retrieval-based experience banks. Ultimately, our results demonstrate that complex agent experience can be packaged into highly transferable, declarative skills—requiring no parameter updates, no external retrieval modules, and utilizing open-source models as small as 35B parameters.[a]

[a]Work in progress.

1. Introduction

LLM-based agents are increasingly rely on *skills* — structured, reusable documents that encode task-solving procedures, domain knowledge, and operational guidelines — to navigate complex environments (Anthropic, 2026b). As these agents are deployed across increasingly broad and nuanced domain-specific use cases, demand for highly specialized skills grows accordingly, creating a scalability bottleneck for manual skill creation and maintenance (Han et al., 2026; Li et al., 2026a; Anthropic, 2026a; Liang et al., 2026). Even when a human-written skill exists, it is not guaranteed to improve performance for a given agent, model, or task distribution (e.g., Table 1 shows that a human-expert-written skill that lifts a 122B agent by +20 pp on SpreadsheetBench-Verified (Ma et al., 2024) actively

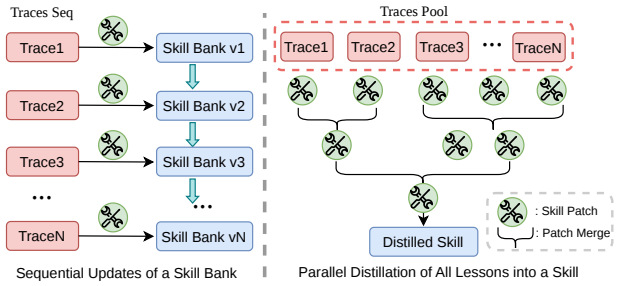

Figure 1 | *Left:* Concurrent work's online setting where lessons from in-coming traces evolve a skill bank sequentially. *Right:* Trace2Skill's analyze a pool of traces in parallel, and hierarchically consolidating lessons to induce generalizable SOPs.

Corresponding author(s): zhoumengyu.zmy@alibaba-inc.com

harms a 35B agent). These pressures motivate automatic *creation* and *adaptation* of skills for specific use cases (Han et al., 2026).

However, synthesizing skills relying solely on an LLM's parametric knowledge yields limited benefits, even with leading proprietary models, primarily because parametric knowledge lacks information about the specifics and common pitfalls of the target domain (Li et al., 2026b; Jiang et al., 2026). To address this, concurrent work proposes improving skills using agent execution experience in an online setting, where an agent continuously interacts with the environment and evolves its skill collection based on incoming trajectories (Yang et al., 2026; Xia et al., 2026a; Alzubi et al., 2026; Zhou et al., 2026; Jiang et al., 2026).

While this continuous, online paradigm has shown promise, **we approach the problem of skill evolution from a different angle—one that more closely mirrors how human experts author skills.** Specifically, we observe that existing online paradigms often diverge from human methodology in two key ways:

- **Skill Fragmentation vs. Consolidation:** Existing works often create new, narrowly tailored skills to host trajectory-local lessons, resulting in massive skill collections that can lead to retrieval difficulties (Li, 2026). In contrast, human experts typically craft a single, comprehensive skill per domain, complete with broad procedural guidance and error prevention checklists.
- **Sequential vs. Holistic Updates:** In an online setting, skills are updated sequentially using lessons from isolated incoming trajectories (Jiang et al., 2026; Xia et al., 2026a). This mimics a scenario where an author continuously edits a skill while sequentially learning about a domain, reacting prematurely before acquiring adequate domain-specific knowledge. Human experts, conversely, build a comprehensive, high-level understanding of the domain before instantiating it into a skill. Figure 1 illustrates these comparisons.

Motivated by these observations, we introduce Trace2Skill, a framework designed to simulate this human, holistic approach. Rather than reacting to trajectories sequentially, Trace2Skill analyzes a wide range of trajectory-local lessons in parallel, and distills common patterns into a single, comprehensive agent skill. Trace2Skill operates in three stages: (1) **Trajectory Generation:** An agent runs in parallel on an evolving set of tasks, producing a pool of execution trajectories. (2) **Parallel Multi-Agent Patch Proposal:** A fleet of success and error-analyst sub-agents independently processes batches of trajectories, proposing targeted patches to the skill. (3) **Conflict-Free Consolidation:** Sub-agent-proposed patches are hierarchically merged into a coherent update to the skill directory, utilizing programmatic conflict detection and format validation at each step.

We process all patches simultaneously during consolidation for two reasons. First, this acts as an inductive reasoning process (Xiong et al., 2025; Li et al., 2025; Lin et al., 2025) that mines generalizable patterns from experience-specific patches, building a high-level understanding of the domain analogous to a human expert's prior knowledge. Second, analyzing a massive number of trajectories in parallel brings substantial efficiency benefits and ensures a holistic view of the domain. This reflects the core design wisdom of agent swarms (Kimi Team, 2026), which process multiple information sources efficiently using parallelized sub-agents. The framework supports two modes: deepening an existing human-written skill, and creating an effective skill from scratch starting from an ineffective LLM-generated draft.

The most surprising finding of this work is not just that trajectory analysis improves skill quality, but that it does so without sacrificing generalizability. Despite the deep analysis over a specific task distribution and trajectories of a specific LLM, evolved skills transfer across model scales (e.g., a skill evolved by Qwen3.5-35B (Team, 2026) improves Qwen3.5-122B) and generalize to out-of-distribution task domains (e.g. from spreadsheet editing to Wikipedia table QA). Analyses attribute this transferability to the successful mining of prevalent, highly useful patterns induced from broad trajectories. This challenges the common assumption that experience is inherently model- and task-specific and must be managed through the retrieval of episodic memories (Ouyang et al., 2026; Wang et al., 2024; Qian et al., 2024; Nottingham et al., 2024; Liu et al., 2025). Instead, we show that experience can be distilled into transferable, declarative skills. We further confirm the effectiveness of Trace2Skill on creating useful skills for math and vision reasoning.

Further analysis shows that Trace2Skill outperforms other popular paradigms of experience-learning: (1) Reasoning Bank (Ouyang et al., 2026) that first saves generalizable lessons from each trajectory, and retrieve useful experiences at inference time based on task similarity. (2) An online setting where new trajectories sequentially come in, and the skill evolves based on new lessons learned. Crucially, because the skills created or deepened by Trace2Skill operate entirely without an external retrieval module, they are seamlessly portable across the broader agent-skill ecosystem.

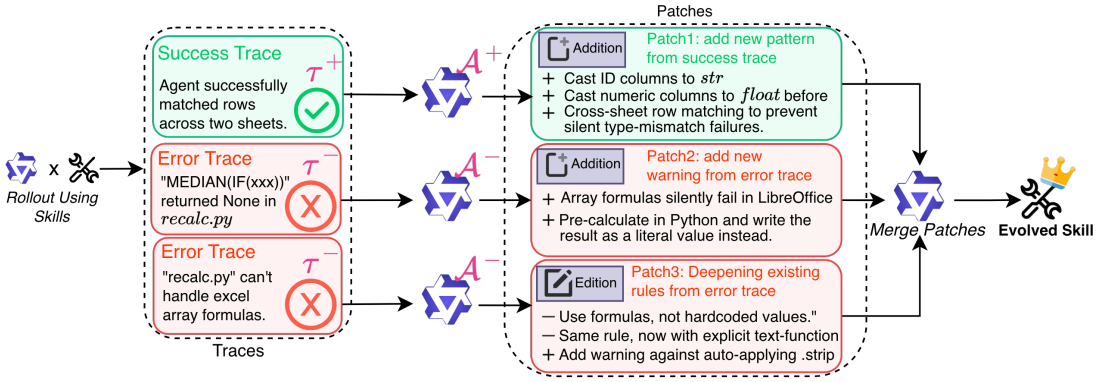

Figure 2 | Overview of Trace2Skill's three-stage pipeline. **Stage 1:** a frozen agent π_θ rolls out on the evolving set using an initial skill $\mathcal{S}_0$ (human-written or LLM-drafted), producing labeled trajectories $\mathcal{T}^-$ (failures) and $\mathcal{T}^+$ (successes). **Stage 2:** parallel error and success analysts independently processes individual traces and proposes skill patches. **Stage 3:** all patches are merged into a single consolidated update via inductive reasoning with programmatic conflict prevention, producing an evolved skill $\mathcal{S}^*$ with improved performance and generalization.

- Trace2Skill, a framework for automatic skill creation and adaptation that supports both deepening existing human-written skills and creating new ones from scratch. By utilizing fully parallelized patch proposal and conflict-free consolidation, Trace2Skill mirrors human skill writing: building broad prior knowledge through extensive trajectory analysis before drafting comprehensive skills (§2).
- Empirical evidence that trajectory-grounded evolution yields high-quality, generalizable skills that transfer effectively across LLM scales and out-of-distribution task domains (§3).
- A demonstration that open-source, small-scale LLMs (e.g., 35B) are sufficient for robust skill evolution, removing the dependency on proprietary models seen in concurrent work (§3).
- Further analysis showing that parallelized outperforms sequential online skill updates; single comprehensive skill outperforms retrieval-based reasoning banks; and agentic error analysis outperforms plain LLM-based analysis (§4).

2. Trace2Skill

Figure 2 visualizes the three-stage pipeline of Trace2Skill. We first formalize the skill structure and the evolution objective (§2.1). Stage 1, Stage 2, and Stage 3 are detailed in §2.2, §2.3, and §2.4 accordingly.

2.1. Skill and Problem Formalization

A *skill* $\mathcal{S}$ is a structured, human-readable knowledge directory consisting of a root markdown document M (`SKILL.md`) and a set of auxiliary resources $\mathcal{R} = \{r_1, \ldots, r_K\}$:

$$\mathcal{S} = (M, \mathcal{R}), \quad \mathcal{R} = \{\text{scripts, references, assets}\}. \tag{1}$$

M encodes procedural knowledge in natural language: when to apply a technique, step-by-step strategies, and known failure modes. Auxiliary resources provide executable scripts for deterministic subtasks and context- or domain-specific references.

Skill Evolution Problem Formalization. Let π_θ denote an LLM-based agent with fixed parameters θ, equipped at inference time with a prepended skill $\mathcal{S}$. Let $\mathcal{D}_{\text{evolve}}$ and $\mathcal{D}_{\text{test}}$ be disjoint task sets drawn from potentially different distributions. We define success rate as

$$\mathcal{P}(\mathcal{S}; \pi_\theta, \mathcal{D}) = \frac{1}{|\mathcal{D}|} \sum_{t \in \mathcal{D}} \mathbf{1}[\pi_\theta(t; \mathcal{S}) = y_t^*], \tag{2}$$

where y_t^* is the ground-truth answer for task t. The objective of *skill evolution* is to construct an improved skill from trajectories on $\mathcal{D}_{\text{evolve}}$, without updating θ, such that:

$$\mathcal{S}^* = \mathcal{E}(\mathcal{S}_0, \mathcal{D}_{\text{evolve}}; \pi_\theta), \quad \mathcal{P}(\mathcal{S}^*;\, \pi_\theta,\, \mathcal{D}_{\text{test}}) > \mathcal{P}(\mathcal{S}_0;\, \pi_\theta,\, \mathcal{D}_{\text{test}}). \tag{3}$$

We study two initializations for $\mathcal{S}_0$: a human-expert-written skill (deepening mode) and an LLM-generated draft from parametric knowledge alone (creation mode), reflecting the two primary real-world use cases of Trace2Skill.

2.2. Stage 1: Trajectory Generation

We adopt ReAct (Yao et al., 2023) as the agent harness. Given $\mathcal{S}_0$, we run π_θ on each task $t_i \in \mathcal{D}_{\text{evolve}}$ with query q_i, yielding a trajectory:

$$\tau_i = \pi_\theta(q_i; \mathcal{S}_0) = \big(q_i,\ (r_1^{(i)}, a_1^{(i)}, o_1^{(i)}),\ \ldots,\ (r_{T_i}^{(i)}, a_{T_i}^{(i)}, o_{T_i}^{(i)}),\ y_i\big), \tag{4}$$

where $r_k^{(i)}$ is the k-th reasoning trace, $a_k^{(i)}$ the tool call, $o_k^{(i)}$ the observation, and $y_i \in \{0, 1\}$ the correctness outcome. The corpus $\mathcal{T} = \{\tau_1, \ldots, \tau_N\}$ is partitioned into:

$$\mathcal{T}^- = \{\tau_i \in \mathcal{T} : y_i = 0\}, \qquad \mathcal{T}^+ = \{\tau_i \in \mathcal{T} : y_i = 1\}. \tag{5}$$

Trajectory generation is fully parallelizable; in practice, 200 trajectories with 50+ turns using a 122B-parameter LLM require less than 2 GPU-hours. The agent system prompt template is reproduced in Appendix B.1.

2.3. Stage 2: Parallel Multi-Agent Patch Proposal

A fleet of specialized analyst sub-agents, each assigned to a *single* trajectory τ_i, independently proposes edits to the skill. Each analyst takes a frozen copy of $\mathcal{S}_0$ and one trajectory, and outputs a *skill patch*:

$$p_i = \begin{cases} \mathcal{A}^-(\mathcal{S}_0,\ \tau_i), & \tau_i \in \mathcal{T}^- \\ \mathcal{A}^+(\mathcal{S}_0,\ \tau_i), & \tau_i \in \mathcal{T}^+ \end{cases} \tag{6}$$

All analysts are dispatched concurrently to a thread pool, yielding the patch pool $\mathcal{P} = \{p_i\}$ with no sequential dependency between agents. Both roles are instructed to propose patches that generalize beyond the single observed trajectory, and strictly follow Anthropic's recommendation for skill writing style (Anthropic, 2026a) on conciseness, actionability, and hierarchical disclosure. Since we assume no stronger teacher model is available, errors are substantially harder to diagnose than successes, motivating asymmetric analyst designs.

Success Analyst ($\mathcal{A}^+$). $\mathcal{A}^+$ follows a fixed single-pass workflow: it cleans the trajectory, identifies generalizable behavior patterns that contributed to the correct answer, and proposes skill patches. The single-call design is both sufficient and efficient since successful trajectories require no interactive diagnosis.

Error Analyst ($\mathcal{A}^-$). $\mathcal{A}^-$ is implemented as a ReAct-style multi-turn agentic loop. Given $\tau_i \in \mathcal{T}^-$, it can inspect the full trace, read input/output files, and compare the agent's answer against ground truth — iteratively narrowing down the root cause before proposing a patch. The loop terminates when $\mathcal{A}^-$ either (1) successfully fixes and causally explains the failure, or (2) exhausts its turn budget. If neither condition yields a valid causal analysis, τ_i is excluded from the patch pool. This quality gate ensures every patch in $\mathcal{P}^-$ is grounded in a verified failure cause, in contrast to prior work deriving insights via a single non-interactive LLM call (Ouyang et al., 2026). An ablation comparing agentic and LLM-only error analysis is presented in §4.3.

Independence of Patch Proposal. All analysts operate on a frozen copy of $\mathcal{S}_0$ with no visibility into other agents' patches. This independence prevents premature convergence, preserving the full diversity of per-trajectory observations in $\mathcal{P}$. Analyst prompt templates and representative example patches are provided in Appendix B.2.

2.4. Stage 3: Conflict-Free Patch Consolidation

Let $\mathcal{P} = \mathcal{P}^- \cup \mathcal{P}^+$ be the full patch pool from Stage 2. Stage 3 consolidates $\mathcal{P}$ into a single coherent skill update p^* and applies it to $\mathcal{S}_0$, jointly serving two purposes: conflict elimination and inductive generalization.

Hierarchical merging with programmatic conflict prevention. The patches are merged in a hierarchy of $L = \lceil \log_{B_{\text{merge}}} |\mathcal{P}| \rceil$ levels ($L \leq L_{\max}$). At each level ℓ, groups of up to B_{merge} patches are synthesized into a single consolidated patch:

$$p^{(\ell+1)} = \mathcal{M}\Big(\pi_\theta,\ \mathcal{S}_0,\ \{p_1^{(\ell)}, \ldots, p_{B_{\text{merge}}}^{(\ell)}\}\Big), \tag{7}$$

where $\mathcal{M}(\pi_\theta, \mathcal{S}_0, \cdot)$ deduplicates, resolves conflicts, and preserves unique insights. Crucially, $\mathcal{M}$ reuses the same π_θ that generated trajectories and proposed patches — making the entire pipeline *self-contained*: a single LLM collects experience, analyzes it, and distills it into an improved skill with no external teacher. The final p^* is translated into diff-style edit operations and applied programmatically. Three deterministic guardrails enforce correctness: (1) patches referencing non-existent files are rejected; (2) edits targeting the same line range within the same file are flagged as conflicts and withheld; (3) the updated $\mathcal{S}$ is validated by a skill format checker.

Patch consolidation as inductive reasoning. Beyond conflict elimination, the hierarchical application of $\mathcal{M}$ performs *inductive reasoning* over the patch pool. Because each p_i derives from a single trajectory, $\mathcal{P}$ as a whole encodes the distribution of behaviors π_θ exhibits across the evolving set. $\mathcal{M}$ is explicitly instructed to identify *prevalent patterns* — edits appearing consistently across independent patches — on the grounds that recurring observations across diverse trajectories are more likely to reflect systematic task properties and generalize to unseen tasks and different agent models. Conversely, edits appearing in only one or few patches are treated as potentially idiosyncratic and discarded. This prevalence-weighted consolidation is the mechanism by which deep per-trajectory analysis produces a generalizable skill.

The evolved skill $\mathcal{S}^* = (M^*, \mathcal{R}^*)$ replaces $\mathcal{S}_0$ and is used directly at inference without any retrieval index. The merge operator prompt template and an example consolidated patch p^* are given in Appendix B.3.

2.5. Two Evolution Modes

Skill deepening. $\mathcal{S}_0$ is initialized with a human-expert-written skill. The pipeline refines $\mathcal{S}_0$ by adding failure-specific guidance from $\mathcal{T}^-$ and reinforcing effective strategies from $\mathcal{T}^+$.

Skill creation from scratch. $\mathcal{S}_0$ is initialized with a skill drafted by π_θ from parametric knowledge alone, with no access to task trajectories. As we show in §3, this draft provides no substantial improvement over no skill — $\mathcal{P}(\mathcal{S}_0; \pi_\theta, \mathcal{D}_{\text{test}}) \approx \mathcal{P}(\emptyset; \pi_\theta, \mathcal{D}_{\text{test}})$ — so evolution from this point constitutes genuine skill creation: the pipeline produces a useful skill from a performance-neutral initialization, driven entirely by trajectory evidence.

3. Experiments

3.1. Experimental Setup

Datasets and Skills. Our main experiments focus on the spreadsheet domain, which challenges agents to interact with a file system and manipulate `xlsx` files whose contents are hard to inspect without structured tooling. We use SpreadsheetBench-Verified (Ma et al., 2024), splitting its 400 samples into 200 for the *evolving set* and 200 held-out for testing; no test samples are seen during evolution. We additionally report Soft (sub-problem pass rate) and Hard (all sub-problems must pass) scores on the full SpreadsheetBench. For out-of-distribution (OOD) generalization, we evaluate on WikiTableQuestions (Pasupat & Liang, 2015) (WikiTQ), which differs in data source (Wikipedia) and task type (compositional semantic parsing); inputs and expected outputs are converted to spreadsheet format so the `xlsx` skill applies without modification. All results are averaged over three random seeds (41, 42, 43) using each benchmark's official evaluation criteria.

Two baseline skills are compared: (1) the Anthropic official `xlsx` skill (**Human-Written**), a high-quality human-expert-written skill; and (2) an `xlsx-basic` skill generated by prompting Qwen3.5-122B-A10B from parametric knowledge alone (**Parametric**), containing only common-sense-level task descriptions with no trajectory grounding (details in Appendix B.1).

Skill Settings. We evaluate six conditions: **No Skill** (no skill document), **Human-Written** (`xlsx`), **Parametric** (`xlsx-basic`), **+Error** (Trace2Skill with error analysts only), **+Success** (Trace2Skill with success analysts only), and **+Combined** (Trace2Skill with both analyst types). *Skill Deepening* initializes from Human-Written; *Skill Creation* initializes from Parametric.

Table 1 | Main results shown as deltas (%). **Skill Author** = model that evolved the skill (row groups); **Skill User** = model at inference (column groups). Reference rows remain absolute scores for context. Evolved rows show signed deltas with per-column color intensity; green = improvement, red = decline. **Deltas:** *Deepening* is measured against the **Human-Written** baseline; *Creation* against the **Parametric** baseline. **Avg**: equally weights in-distribution SpreadsheetBench (Vrf/Soft/Hard, both model scales) and OOD WikiTQ (both model scales), expressed as delta from the corresponding baseline.

	Skill User: Qwen3.5-122B-A10B				*Skill User: Qwen3.5-35B-A3B*				
	SpreadsheetBench			*OOD*	*SpreadsheetBench*			*OOD*	
Condition	**Vrf↑**	**Soft↑**	**Hard↑**	**WikiTQ↑**	**Vrf↑**	**Soft↑**	**Hard↑**	**WikiTQ↑**	**Avg↑**
Reference (absolute scores)									
No Skill	27.67	28.90	17.57	21.50	19.00	18.00	4.60	13.33	18.35
Human-Written	48.33	36.30	17.03	74.68	9.67	13.03	3.37	9.02	31.57
Parametric	26.17	36.60	17.50	23.73	20.17	13.70	3.87	20.14	20.80
Skill Author: Qwen3.5-122B-A10B									
Deepening (init: Human-Written)									
+Error	+17.50	+10.30	+10.40	+1.62	**+27.00**	+9.44	+2.86	+9.26	+9.18
+Success	-21.83	-8.57	+0.04	-10.35	+9.16	+3.57	+1.56	+12.09	-0.90
+Combined	**+21.50**	**+10.87**	**+12.50**	+4.56	+21.16	+8.84	+1.80	+6.64	+9.19
Creation (init: Parametric)									
+Error	+22.83	+3.77	+5.87	+7.89	+8.66	+9.53	+4.00	+2.06	+7.04
+Success	+15.33	-0.93	+4.33	+23.70	+12.83	**+11.57**	**+6.13**	+30.36	+17.62
+Combined	+0.16	-9.23	-1.40	+32.32	-1.17	+3.73	+1.36	+29.70	+14.96
Skill Author: Qwen3.5-35B-A3B									
Deepening (init: Human-Written)									
+Error	+16.67	+8.50	+8.14	-6.36	+17.33	+9.17	+4.83	+2.71	+4.47
+Success	-22.00	-8.83	-0.50	+1.46	+11.00	+3.64	+0.83	**+43.23**	+9.85
+Combined	+6.67	+3.87	+4.17	+2.65	+20.00	+5.77	+2.36	+42.20	**+14.78**
Creation (init: Parametric)									
+Error	+1.00	-7.70	+1.03	**+57.65**	+3.83	+7.30	+2.66	+12.66	+18.26
+Success	+5.33	-4.57	+2.43	+9.09	+5.66	+5.80	+2.63	+3.31	+4.54
+Combined	-0.84	-9.17	-1.63	+30.82	-0.17	+4.40	+1.26	+18.00	+11.69

Implementation Details. Trace2Skill conducts end-to-end self-evolution: the same LLM serves as trajectory generator, patch proposer, and skill editor. We experiment with two Qwen3.5 MoE models: Qwen3.5-122B-A10B and Qwen3.5-35B-A3B. Both are instruct/think hybrid models; we use instruct mode for multi-turn ReAct-style agentic tasks and thinking mode for single-call tasks (hierarchical merging, success analysis, patch conversion). Models are served with vLLM (Kwon et al., 2023) using the recommended Qwen3.5 generation configuration[1]. Stage 1 generates 1 trajectory per problem. At Stage 2, 128 sub-agents run in parallel, and we use a merge batch size of 32. For all ReAct-style agents, we set the interaction turn budget to 100.

3.2. Main Results

Table 1 presents results across all skill conditions, model scales, and transfer directions. We report the performance of evolved skills as deltas against their respective baselines: comparing skill deepening against existing human-written skills, and skill creation against the model's base parametric performance. We use **Avg** as the primary summary metric: a skill that genuinely benefits an agent should transfer across model scales *and* task domains, so Avg equally weights in-distribution SpreadsheetBench performance (Vrf/Soft/Hard, both model scales) and WikiTQ transfer performance (both model scales), rewarding generalization rather than in-distribution specialization.

[1]See https://huggingface.co/Qwen/Qwen3.5-35B-A3B and https://huggingface.co/Qwen/Qwen3.5-122B-A10B.

Human-written is a strong handcrafted prior, but not a portable one; parametric is weak. The Human-Written baseline is strong for the 122B agent, reaching 48.33% on SprBench-Vrf and 74.68% on WikiTQ, but it does not transfer cleanly across model scale: for the 35B agent it underperforms No Skill by −9.3 pp on SprBench-Vrf and −4.3 pp on WikiTQ. By contrast, the Parametric baseline remains close to No Skill overall (26.17% vs. 27.67% SprBench-Vrf for the 122B agent), confirming that parametric knowledge alone does not yield useful skill content (Han et al., 2026). These two references motivate both Deepening and Creation: the former asks whether a strong manual prior can be refined, while the latter asks whether trajectory-grounded distillation can build a useful skill starting from an inadequate one.

Deepening reliably strengthens the human-written skill on in-distribution spreadsheet tasks. Starting from Human-Written, 122B-authored Deepening gains +17.5 pp on SprBench-Vrf with +Error and +21.5 pp with +Combined, while also improving Soft and Hard scores. These gains are not confined to the authoring model: the 35B-authored Deepening +Error skill gains +16.7 pp on SprBench-Vrf when used by the 122B agent, and the 122B-authored counterpart gains +27.0 pp for the 35B agent. The same refined skills also transfer beyond the training distribution, with 122B-authored Deepening improving WikiTQ by +1.6 pp (+Error) and +4.6 pp (+Combined).

Creation substantially outperforms the weak parametric baseline and can match or exceed human-written quality. Because Parametric is a poor starting point, the relevant comparison is whether distilled skills recover meaningful capability from scratch. The answer is yes: 122B-authored Creation +Error gains +22.8 pp on SprBench-Vrf, bringing performance close to Human-Written despite starting from a weak prior. In some settings Creation even outperforms Human-Written: the 35B-authored Creation +Error skill, when used by the 122B agent, gains +57.7 pp on WikiTQ over Parametric (**best in table**, reaching 81.38%) and surpasses Human-Written by +6.7 pp.

The Avg column identifies the settings that are robust across in-distribution, OOD, and cross-model use. Viewed through Avg, the strongest configurations are those that improve multiple slices of the table at once rather than spiking on only one benchmark or skill user model. The best Avg scores come from Creation, with 35B-authored Creation +Error reaching +18.3 pp and 122B-authored Creation +Success reaching +17.6 pp, showing that from-scratch skill synthesis can remain strong after averaging over both datasets and both user models. At the same time, Deepening remains broadly competitive, and +Combined is noteworthy because it stays consistently high across all four Author–Mode combinations rather than depending on a single especially favorable setting.

Across analyst types, +Combined is the most consistently strong signal, +Error the most reliable, and +Success the most volatile. Measured by Avg improvement over the corresponding reference baseline, **+Combined** is the steadiest high performer, while **+Error** remains reliably positive in every setting and serves as the safest default signal. **+Success**, by contrast, has the highest variance: it produces the largest single-setting Avg gain (+17.6 pp for 122B-authored Creation) but is also the only condition that drops below baseline (−0.9 pp for 122B-authored Deepening). This pattern suggests that success-derived patches can be highly valuable, but only when the hierarchical merge filters them effectively; otherwise they are less stable than error-driven updates, motivating a more selective success analyst design (see §6).

3.3. Math Reasoning

To assess whether Trace2Skill generalizes beyond spreadsheets, we apply it to mathematical reasoning using **DAPO-Math-Train-400** as the evolving set.

We evaluate on **DAPO-Math-Test-100** (in-distribution; pass rate %) and **AIME 2026** (out-of-distribution competition mathematics; avg@8 over 30 problems). Following the cross-model protocol of Section 3.2, we create skills from scratch using error analysts. Results are shown in Table 2.

Table 2 | Math reasoning results shown as deltas from the No Skill baseline. **D-Test**: DAPO-Math-Test-100 pass rate (%); **AIME**: AIME 2026 avg@8 over 30 problems (%). Reference row remains absolute; evolved rows use green / red delta intensity.

	Skill User: 122B		*Skill User: 35B*	
Condition	**D-Test↑**	**AIME↑**	**D-Test↑**	**AIME↑**
No Skill	92.0	90.4	89.0	83.3
122B-Authored +Error	**+3.0**	**+2.9**	**+5.0**	**+5.0**
35B-Authored +Error	**+2.0**	**+1.3**	**+4.0**	**+0.5**

Distilled math skills yield consistent gains across models and benchmarks (Table 2). The 122B-authored skill gains +3.0 pp on DAPO-Math-Test-100 and +2.9 pp on AIME 2026 (same-model), and transfers positively cross-model: +5.0 pp on DAPO-Math-Test-100 and +5.0 pp on AIME 2026 for the 35B agent. The 35B-authored skill is comparably effective: +4.0 pp same-model and +2.0 pp cross-model on DAPO-Math-Test-100, confirming that trajectory-grounded distillation is domain-agnostic and scales to competition-level evaluation.

3.4. Visual Question Answering

To evaluate whether Trace2Skill generalizes to multimodal visual reasoning, we apply it to **Visual Question Answering (VQA)** using **DocVQA** Mathew et al. (2020) as the target benchmark. DocVQA requires jointly understanding document images—forms, tables, invoices, letters, reports—and answering natural-language questions by extracting, locating, and reasoning over visual and textual elements. We use the official validation split (5,349 question–image pairs), reserving the first 2,700 instances as the *evolving set* and the remaining 2,649 as the held-out *evaluation set*. We report **ANLS** (Average Normalized Levenshtein Similarity, the official metric) and **Accuracy** (ANLS $\geq$ 0.5, %). Similar to Section 3.2, we create skills from scratch using error analysts. Results are shown in Table 3.

Table 3 presents a nuanced picture. The No Skill row reveals an unexpected reversal: the 35B agent (0.6843 ANLS) outperforms the 122B agent (0.6424 ANLS) on DocVQA without any skill.

Table 3 | DocVQA results shown as deltas from the No Skill baseline (evaluation set: 2,649 instances). **ANLS**: Average Normalized Levenshtein Similarity; **Acc**: ANLS $\geq$ 0.5 (%). Reference row remains absolute; evolved rows use green / red delta intensity.

	Skill User: 122B		*Skill User: 35B*	
Condition	**ANLS↑**	**Acc↑**	**ANLS↑**	**Acc↑**
No Skill	0.6424	71.2	0.6843	75.2
122B-Authored +Error	**+0.1639**	**+15.3**	**+0.1554**	**+13.6**
35B-Authored +Error	+0.0093	+0.9	-0.0620	-6.2

Despite this task performance advantage, the 35B model falls behind as a skill *author*. The 35B-authored skill yields negligible gains for the 122B model (+0.009 ANLS) and actively degrades same-model 35B performance (−0.062 ANLS, −6.2 pp accuracy). By contrast, the 122B-authored skill gains +0.1639 ANLS and +15.3 pp accuracy (same-model), and transfers just as strongly to the 35B model (+0.1554 ANLS, +13.6 pp accuracy). This dissociation suggests that inductive reasoning for skill authoring — identifying recurring failure patterns across trajectories and distilling them into actionable rules — is a distinct capability from task execution. A model that performs well on DocVQA does not necessarily possess the reflective capacity to analyze *why* it fails and generalize those observations into a transferable skill.

4. Analysis

4.1. Parallel Consolidation vs. Sequential Editing

In the online skill evolution paradigm, the skill is updated sequentially as new trajectory batches arrive. We isolate the contribution of parallel, many-to-one consolidation by comparing against two sequential baselines: **Seq-**B=1, where the skill is updated after every single trajectory, and **Seq-**B=4, where the skill is updated after every batch of four trajectories. All three conditions use error analysts only and initialize from the Human-Written skill; results are shown in Table 4.

On the 122B model, parallel consolidation outperforms both sequential settings across all SpreadsheetBench metrics (+4.0 pp Vrf over Seq-B=1, +6.8 pp over Seq-B=4). On the 35B model, parallel wins on Vrf but Seq-B=1 scores modestly higher on Soft and Hard, suggesting the smaller model may benefit from more incremental updates in some conditions. However, this marginal quality variation comes at 20× the computational cost, and the efficiency gap widens linearly with the number of trajectories analyzed.

Latency. With W=128 workers and $N\approx$70 error lessons, all analysts execute in a single parallel round; the hierarchical merge adds $\lceil \log_2 N \rceil \approx 7$ further sequential rounds (one per merge layer), yielding ≈8 sequential

Table 4 | Parallel consolidation vs. sequential editing on SpreadsheetBench (same-model Deepening, +Error only, %). Seq-B: skill updated after every batch of B trajectories. **Bold** = best per column.

	Skill User: 122B			*Skill User: 35B*			
Condition	**Vrf↑**	**Soft↑**	**Hard↑**	**Vrf↑**	**Soft↑**	**Hard↑**	**Time↓**
Seq-B=4	59.00	40.63	20.63	26.17	22.37	7.47	~15 min
Seq-B=1	61.83	44.40	25.40	26.00	**23.83**	**10.57**	~60 min
Parallel (ours)	**65.83**	**46.60**	**27.43**	**27.00**	22.20	8.20	**~3 min**

LLM-call rounds in total. The sequential baselines require N and $\lceil N/B \rceil$ rounds respectively, since each skill edit depends on the preceding one. In practice this translates to 3 min (parallel) vs. 60 min (Seq-B=1, 20×) and 15 min (Seq-B=4, 5×), with the gap scaling linearly in N. All times are in node hours of an 8-GPU A800 node.

Beyond efficiency, parallel consolidation has a structural advantage: all patch proposals are derived from the same frozen initial skill $\mathcal{S}_0$, preventing the *sequential drift* inherent to the online setting, where each skill update alters the context in which subsequent trajectories are analyzed. The hierarchical merge then performs inductive reasoning over the full population of trajectory-local observations simultaneously, selecting patterns that recur across diverse trajectories rather than patterns that recur in the most recent updates.

4.2. Trace2Skill vs. Retrieval-Memory Baseline

ReasoningBank (Ouyang et al., 2026) stores generalizable lessons derived from each trajectory and retrieves the most relevant memories at inference time using task-query similarity. Since it draws on both success and failure trajectories, we compare it against **+Combined**, which uses the same trajectory pool but distills it into a portable skill document rather than maintaining a retrieval index. We implement a ReasoningBank-style retrieval baseline with their official prompt and recommended retrieval setting (top 1) with Qwen3-Embedding-8B as the retriever. Results on same-model Deepening are shown in Table 5.

Table 5 | Trace2Skill (Human Written+Combined) vs. ReasoningBank on SpreadsheetBench (same-model Deepening, %). ReasoningBank retrieves success and failure memories at inference via Qwen3-Embedding-8B; +Combined distills the same trajectory pool into a single portable skill with no retrieval module. **Bold** = best per column.

	Skill User: 122B			*Skill User: 35B*		
Condition	**Vrf↑**	**Soft↑**	**Hard↑**	**Vrf↑**	**Soft↑**	**Hard↑**
ReasoningBank (Ouyang et al., 2026)	56.00	40.10	21.30	20.50	17.30	4.97
Human-Written+Combined (ours)	**69.83**	**47.17**	**29.53**	**29.67**	**18.80**	**5.73**

+Combined outperforms ReasoningBank by large margins: +13.8 pp Vrf, +7.1 pp Soft, +8.2 pp Hard for the 122B model; +9.2 pp Vrf, +1.5 pp Soft, +0.8 pp Hard for 35B. ReasoningBank on the 35B model (20.50% Vrf) barely exceeds No Skill (19.00%), indicating the retriever fails to surface relevant guidance when the 35B model's query representations do not align well with the stored memory embeddings.

We attribute the performance gap to three factors. First, retrieval quality is sensitive to surface-level similarity between the test query and stored memory keys: when the test distribution differs in phrasing or structure from the evolving set, retrieval degrades, while the distilled skill transfers without modification. Second, retrieved snippets compete with the task context for model attention, whereas a skill pre-loaded into the system prompt is already integrated before any task-specific content is seen. Third, Trace2Skill's hierarchical merge actively deduplicates and abstracts trajectory-local observations into general principles; raw trajectory summaries in a retrieval bank are not filtered for redundancy or generalizability. Taken together, these results support the premise that distillation into a compact, model-agnostic skill document is a more effective use of trajectory evidence than episodic retrieval.

4.3. Agentic Error Analysis vs. Single-LLM-Call Baselines

Many prior works derive transferable lessons or skills from error trajectories via a single non-interactive LLM call (Ouyang et al., 2026; Xia et al., 2026a; Yang et al., 2026; Jiang et al., 2026). The **+Error LLM** condition ablates our agentic loop design: a single LLM call receives each error trajectory and proposes a patch directly, without the ability to inspect files, query ground truth, or iteratively narrow the root cause. Table 6 compares +Error (agentic, ours) against +Error LLM across all four Author–Mode combinations.

Table 6 | Agentic error analysis (+Error) vs. single-LLM-call error analysis (+Error LLM) across all Author–Mode combinations (%). Same-model cells are shaded; plain cells are cross-model transfer. **Bold** = better of the two conditions per cell.

	Skill User: Qwen3.5-122B-A10B				*Skill User: Qwen3.5-35B-A3B*				
	SpreadsheetBench			*OOD*	*SpreadsheetBench*			*OOD*	
Condition	**Vrf**	**Soft**	**Hard**	**WikiTQ**	**Vrf**	**Soft**	**Hard**	**WikiTQ**	**Avg**
Skill Author: Qwen3.5-122B-A10B									
Deepening									
+Error (ours)	65.83	**46.60**	**27.43**	**76.30**	**36.67**	**22.47**	**6.23**	**18.28**	**40.75**
+Error LLM	**67.00**	43.93	25.23	39.81	25.00	22.43	6.23	11.24	28.58
Creation									
+Error (ours)	**49.00**	**40.37**	**23.37**	31.62	**28.83**	**23.23**	**7.87**	22.20	**27.84**
+Error LLM	27.17	27.73	16.20	**47.26**	19.83	17.60	4.70	**23.30**	27.08
Skill Author: Qwen3.5-35B-A3B									
Deepening									
+Error (ours)	**65.00**	**44.80**	**25.17**	68.32	27.00	**22.20**	8.20	**11.73**	**36.04**
+Error LLM	37.83	22.93	12.83	**77.05**	**30.50**	20.17	**8.73**	9.95	32.83
Creation									
+Error (ours)	**27.17**	**28.90**	**18.53**	**81.38**	**24.00**	**21.00**	**6.53**	**32.80**	**39.06**
+Error LLM	22.00	27.67	16.60	54.61	23.50	16.87	4.93	11.24	25.76

Agentic analysis wins in Avg across all settings. Agentic +Error outperforms +Error LLM in weighted Avg in all four settings, with gaps of +12.2 pp (122B Deepening), +0.8 pp (122B Creation), +3.2 pp (35B Deepening), and +13.3 pp (35B Creation). The 122B Creation setting is the only near-tie: both conditions score around 27 pp, masking a striking internal divergence (see below).

Agentic error analysis produces more transferable patches. The Avg advantage holds across both in-distribution SpreadsheetBench and OOD WikiTQ columns: +Error LLM patches that achieve comparable ID scores frequently degrade on WikiTQ and cross-model cells, while agentic patches remain positive across both axes.

Why the agentic loop produces more transferable patches. A qualitative study of 33 shared error cases confirms the structural advantage built into $\mathcal{A}^-$ (§2): the two pipelines reach strong agreement on only 4 cases (12.1%), with clear disagreement in 18 (54.5%). The LLM-only analyzer, limited to the execution log, over-attributes parse failures as the primary root cause in 57% of cases where parse-error messages appear (vs. 14% for agentic), and in at least one case hallucinated three distinct failure causes for a trajectory where artifact evaluation confirmed the output was already correct. Artifact access and fix validation let $\mathcal{A}^-$ reject such false positives and anchor each patch to a verified failure mechanism — producing the domain-general guardrails that transfer across model scales and OOD settings.

4.4. Generalizable SoPs Learned

We inspect the 323 map patches produced by the 122B Deepening +Combined run to characterize which standard operating procedures (SoPs) the pipeline distills. The four most prevalent themes together account for 546/323 patch citations (patches can cite multiple themes) and are encoded directly in the main `SKILL.md`; less common patterns are detailed in Appendix A.

Formula recalculation and write-back verification (178/323 patches). Run `recalc.py` after every formula write and reopen with `data_only=True` to confirm evaluation; skipping this step leaves cells stale and is the single most common error mode in the run.

Tool selection: `openpyxl` over `pandas.to_excel()` (177/323 patches). Use `pandas` for read/transform logic and `openpyxl` for write-back; copy the input file to the output path first to preserve all structural anchors. `pandas.to_excel()` silently destroys formula relationships and named ranges.

Explicit read-back verification (138/323 patches). After writing, reopen the output file and confirm every target cell holds the expected value before submitting; error trajectories that fail characteristically omit this check.

Structural-edit safety (53/323 patches). Delete rows in descending order to prevent index-shift corruption; copy the input workbook before editing to preserve formatting and formulas. Error trajectories document both failure modes; success trajectories confirm the protective workflow.

Niche quirks are routed to `references/`. Low-support observations are not discarded but routed into 13 supplementary reference files rather than the main `SKILL.md`. For example, cell color extraction, FIFO vs LIFO mismatch under a special business logic etc. This mirrors established skill-design practice: procedural guidance flows from general to case-specific, with the main document encoding universal workflow rules and `references/` serving as an on-demand look-up layer for infrequent edge cases. Trace2Skill recovers this hierarchy automatically from trajectory evidence rather than requiring manual curation.

5. Related Work

Agent Skills. Anthropic's skill framework formalizes skills as lightweight, expert-authored documents that encode standard operating procedures (SOPs) for focused task domains. These are designed for dynamic loading, progressive disclosure, and compatibility with diverse agent harnesses (Anthropic, 2026b). SkillsBench (Li et al., 2026b) provides the first systematic benchmark of skill quality, revealing that while curated, human-written skills generally improve performance, self-generated skills relying purely on parametric knowledge rarely help. Furthermore, they find that a small set of focused skills consistently outperforms a single, bloated document. Similarly, Li (2026) demonstrate that single agents augmented with in-depth skills can match the performance of multi-agent frameworks, though skill retrieval remains a bottleneck. SWE-Skills-Bench (Han et al., 2026) evaluates skill injection in real software-engineering tasks, reporting an average +1.2% gain when skills are well-matched to the task context, but a notable performance drop during context mismatch. AgentSkillOS and SkillNet extend this ecosystem to encompass skill selection and governance (Li et al., 2026a; Liang et al., 2026). *Our position:* We build upon the established consensus that high-quality, focused skills are critical. However, we address a narrower, underexplored question: given a single skill of bounded scope, how much can systematic trajectory analysis improve it? The vulnerability to context mismatch highlighted by SWE-Skills-Bench directly motivates our design choice to inductively distill generalizable patterns rather than overfit to specific queries.

Experience Memory for Agent Self-Evolution. Early work demonstrated that iterative feedback on execution trajectories can significantly improve agent behavior. For instance, Voyager (Wang et al., 2023) accumulates reusable skills through open-ended interaction, while Reflexion (Shinn et al., 2023) refines decisions via verbal self-reflection on past successes and failures. Building on these foundations, subsequent research has focused on storing trajectory-derived insights in retrieval banks, which are then queried to augment future tasks (Ouyang et al., 2026; Fang et al., 2026; Wang et al., 2024; Qian et al., 2024; Nottingham et al., 2024; Liu et al., 2025). *Our position:* While these systems rely on test-time retrieval from episodic memory banks, we explore a fundamentally different approach: distilling experience into declarative skills as static, shareable artifacts. This distinction is motivated by two properties. First, inductive compression across many trajectories smooths out the quirks of any single episode, yielding robust principles rather than localized anecdotes. Second, a distilled declarative skill is architecture-agnostic and seamlessly shareable across agents and model scales, whereas retrieval banks are typically tightly coupled to the specific harness that generated them.

Automatic Skill Self-Evolution. The closest neighbors to our work automatically evolve skills from agent trajectories. SkillWeaver (Zheng et al., 2025) generates web API skills through structured exploration. AutoSkill (Yang et al., 2026) creates and updates skills online from user chat trajectories via an extraction–maintenance–reuse lifecycle. XSkill (Jiang et al., 2026) maintains dual stores: skills encoding task-level SOPs, and experiences encoding context-sensitive, action-level guidance. EvoSkill (Alzubi et al., 2026) iteratively diagnoses failures and validates skill updates, making it the closest single-system neighbor. Memento-Skills (Zhou et al., 2026) employs stateful markdown skills updated incrementally through a read–write loop. Beyond fully automated evolution, Anthropic's skill-creator system (Anthropic, 2026a) represents the state-of-the-art in human-guided refinement, where practitioners qualitatively revise skills based on agent outputs from a small test set. *Our position:* While prior work has actively explored trajectory-based skill evolution, Trace2Skill introduces three critical differentiators. (1) **Many-to-one consolidation:** We merge all trajectory-local patches simultaneously rather than editing the skill sequentially per trajectory, avoiding order dependence and overfitting to early observations. (2) **Comprehensive declarative artifacts:** We target unified, Anthropic-style skill directories rather than narrow API objects, dual stores, or retrieval-augmented hybrids. (3) **No test-time retrieval:** The evolved skill is consumed directly, making it natively compatible with any agent harness. Furthermore, while concurrent automated systems rely on proprietary LLMs (e.g., Claude), our full pipeline achieves robust evolution using open-source models as small as 35B parameters. Finally, Trace2Skill serves as a scalable complement to manual oversight (like Anthropic's skill-creator), distilling lessons across hundreds of trajectories where human review would bottleneck.

Skill and Policy Co-evolution. SkillRL (Xia et al., 2026a) co-evolves skills and model policies via reinforcement learning, treating skills as localized experience triggers ("when X, do Y") rather than comprehensive SOPs. Similarly, ARISE and MetaClaw (Xia et al., 2026b; Li et al., 2026c) explore dual-timescale online adaptation with continual policy updates. *Our position:* In contrast to co-evolution methods that require parameter updates, we strictly study frozen-model, training-free, artifact-level adaptation, ensuring our distilled skills remain entirely model-agnostic.

6. Conclusion

We introduced Trace2Skill, a framework for automatic skill creation and adaptation that simulates how human experts author skills: accumulating broad domain knowledge through extensive experience before instantiating it into a concise, declarative artifact. Rather than updating a skill sequentially as individual trajectories arrive, Trace2Skill dispatches a parallel fleet of analyst sub-agents to propose targeted editing patches from disjoint trajectory batches. It then consolidates all proposals simultaneously into a single, coherent skill directory via inductive reasoning and programmatic conflict prevention.

Our findings show that skills distilled from one model's trajectories generalize remarkably well across model scales and to out-of-distribution tasks. Furthermore, analyses demonstrate that consolidating a broad set of trajectory-local lessons simultaneously improves both computational efficiency and downstream performance, and a single portable skill folder outperforms retrieval-based per-case experience. By structuring the output as a hierarchical directory (e.g., broad principles in a primary 'SKILL.md' file and case-specific heuristics in a 'references/' subdirectory), Trace2Skill successfully encodes both generalizable patterns and nuanced, case-specific pitfalls.

Limitation and Future Work. As a work in progress, the current paper is limited in these aspects, which we plan to address in the near future: (1) **Causal effect quantification of editing patches:** Currently, patches are consolidated holistically, making it difficult to isolate the marginal contribution or potential interference of any single proposed change. We aim to develop methods to rigorously quantify the causal impact of individual trajectory-derived patches on the final skill. (2) **Tracing the utility of specific skill sections:** We have not yet implemented a mechanism to dynamically trace how heavily the agent relies on specific sections of the generated skill directory during inference. Future work will focus on fine-grained attribution tracking to determine the exact utility of different components (e.g., specific checklist items vs. reference files), which will enable automated pruning of ineffective or distracting skill sections.

References

Salaheddin Alzubi, Noah Provenzano, Jaydon Bingham, Weiyuan Chen, and Tu Vu. Evoskill: Automated skill discovery for multi-agent systems, 2026. URL https://arxiv.org/abs/2603.02766.

Anthropic. How to create a skill with claude through conversation. Claude Tutorials, 2026a. URL https://claude.com/resources/tutorials/how-to-create-a-skill-with-claude-through-conversation.

Anthropic. What are skills? Claude Help Center, 2026b. URL https://support.claude.com/en/articles/12512176-what-are-skills. Access Date: 2026-03-22.

Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, and Ningyu Zhang. Memp: Exploring agent procedural memory, 2026. URL https://arxiv.org/abs/2508.06433.

Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu. Swe-skills-bench: Do agent skills actually help in real-world software engineering?, 2026. URL https://arxiv.org/abs/2603.15401.

Guanyu Jiang, Zhaochen Su, Xiaoye Qu, and Yi R. Fung. Xskill: Continual learning from experience and skills in multimodal agents, 2026. URL https://arxiv.org/abs/2603.12056.

Kimi Team. Kimi introduces agent swarm: Let 100 ai agents work for you. Kimi Blog, 2026. URL https://www.kimi.com/blog/agent-swarm.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention, 2023. URL https://arxiv.org/abs/2309.06180.

Hao Li, Chunjiang Mu, Jianhao Chen, Siyue Ren, Zhiyao Cui, Yiqun Zhang, Lei Bai, and Shuyue Hu. Organizing, orchestrating, and benchmarking agent skills at ecosystem scale, 2026a. URL https://arxiv.org/abs/2603.02176.

Jiachun Li, Pengfei Cao, Zhuoran Jin, Yubo Chen, Kang Liu, and Jun Zhao. Mirage: Evaluating and explaining inductive reasoning process in language models, 2025. URL https://arxiv.org/abs/2410.09542.

Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, Shuyi Wang, Binxu Li, Qunhong Zeng, Di Wang, Xuandong Zhao, Yuanli Wang, Roey Ben Chaim, Zonglin Di, Yipeng Gao, Junwei He, Yizhuo He, Liqiang Jing, Luyang Kong, Xin Lan, Jiachen Li, Songlin Li, Yijiang Li, Yueqian Lin, Xinyi Liu, Xuanqing Liu, Haoran Lyu, Ze Ma, Bowei Wang, Runhui Wang, Tianyu Wang, Wengao Ye, Yue Zhang, Hanwen Xing, Yiqi Xue, Steven Dillmann, and Han chung Lee. Skillsbench: Benchmarking how well agent skills work across diverse tasks, 2026b. URL https://arxiv.org/abs/2602.12670.

Xiaoxiao Li. When single-agent with skills replace multi-agent systems and when they fail, 2026. URL https://arxiv.org/abs/2601.04748.

Yu Li, Rui Miao, Zhengling Qi, and Tian Lan. Arise: Agent reasoning with intrinsic skill evolution in hierarchical reinforcement learning, 2026c. URL https://arxiv.org/abs/2603.16060.

Yuan Liang, Ruobin Zhong, Haoming Xu, Chen Jiang, Yi Zhong, Runnan Fang, Jia-Chen Gu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, Xin Xu, Tongtong Wu, Kun Wang, Yang Liu, Zhen Bi, Jungang Lou, Yuchen Eleanor Jiang, Hangcheng Zhu, Gang Yu, Haiwen Hong, Longtao Huang, Hui Xue, Chenxi Wang, Yijun Wang, Zifei Shan, Xi Chen, Zhaopeng Tu, Feiyu Xiong, Xin Xie, Peng Zhang, Zhengke Gui, Lei Liang, Jun Zhou, Chiyu Wu, Jin Shang, Yu Gong, Junyu Lin, Changliang Xu, Hongjie Deng, Wen Zhang, Keyan Ding, Qiang Zhang, Fei Huang, Ningyu Zhang, Jeff Z. Pan, Guilin Qi, Haofen Wang, and Huajun Chen. Skillnet: Create, evaluate, and connect ai skills, 2026. URL https://arxiv.org/abs/2603.04448.

Brian S. Lin, Jiaxin Yuan, Zihan Zhou, Shouli Wang, Shuo Wang, Cunliang Kong, Qi Shi, Yuxuan Li, Liner Yang, Zhiyuan Liu, and Maosong Sun. On llm-based scientific inductive reasoning beyond equations, 2025. URL https://arxiv.org/abs/2509.16226.

Yitao Liu, Chenglei Si, Karthik Narasimhan, and Shunyu Yao. Contextual experience replay for self-improvement of language agents, 2025. URL https://arxiv.org/abs/2506.06698.

Zeyao Ma, Bohan Zhang, Jing Zhang, Jifan Yu, Xiaokang Zhang, Xiaohan Zhang, Sijia Luo, Xi Wang, and Jie Tang. Spreadsheetbench: Towards challenging real world spreadsheet manipulation, 2024. URL https://arxiv.org/abs/2406.14991.

Minesh Mathew, Dimosthenis Karatzas, R Manmatha, and CV Jawahar. Docvqa: A dataset for vqa on document images. corr abs/2007.00398 (2020). *arXiv preprint arXiv:2007.00398*, 2020.

Kolby Nottingham, Bodhisattwa Prasad Majumder, Bhavana Dalvi Mishra, Sameer Singh, Peter Clark, and Roy Fox. Skill set optimization: Reinforcing language model behavior via transferable skills, 2024. URL https://arxiv.org/abs/2402.03244.

Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T. Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. Reasoningbank: Scaling agent self-evolving with reasoning memory, 2026. URL https://arxiv.org/abs/2509.25140.

Panupong Pasupat and Percy Liang. Compositional semantic parsing on semi-structured tables, 2015. URL https://arxiv.org/abs/1508.00305.

Cheng Qian, Shihao Liang, Yujia Qin, Yining Ye, Xin Cong, Yankai Lin, Yesai Wu, Zhiyuan Liu, and Maosong Sun. Investigate-consolidate-exploit: A general strategy for inter-task agent self-evolution, 2024. URL https://arxiv.org/abs/2401.13996.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023. URL https://arxiv.org/abs/2303.11366.

Qwen Team. Qwen3.5: Accelerating productivity with native multimodal agents, February 2026. URL https://qwen.ai/blog?id=qwen3.5.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models, 2023. URL https://arxiv.org/abs/2305.16291.

Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory, 2024. URL https://arxiv.org/abs/2409.07429.

Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning, 2026a. URL https://arxiv.org/abs/2602.08234.

Peng Xia, Jianwen Chen, Xinyu Yang, Haoqin Tu, Jiaqi Liu, Kaiwen Xiong, Siwei Han, Shi Qiu, Haonian Ji, Yuyin Zhou, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. Metaclaw: Just talk – an agent that meta-learns and evolves in the wild, 2026b. URL https://arxiv.org/abs/2603.17187.

Chenfei Xiong, Jingwei Ni, Yu Fan, Vilém Zouhar, Donya Rooein, Lorena Calvo-Bartolomé, Alexander Hoyle, Zhijing Jin, Mrinmaya Sachan, Markus Leippold, Dirk Hovy, Mennatallah El-Assady, and Elliott Ash. Co-DETECT: Collaborative discovery of edge cases in text classification. In Ivan Habernal, Peter Schulam, and Jörg Tiedemann (eds.), *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 354–364, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-334-0. doi: 10.18653/v1/2025.emnlp-demos.25. URL https://aclanthology.org/2025.emnlp-demos.25/.

Yutao Yang, Junsong Li, Qianjun Pan, Bihao Zhan, Yuxuan Cai, Lin Du, Jie Zhou, Kai Chen, Qin Chen, Xin Li, Bo Zhang, and Liang He. Autoskill: Experience-driven lifelong learning via skill self-evolution, 2026. URL https://arxiv.org/abs/2603.01145.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2023. URL https://arxiv.org/abs/2210.03629.

Boyuan Zheng, Michael Y. Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, and Yu Su. Skillweaver: Web agents can self-improve by discovering and honing skills, 2025. URL https://arxiv.org/abs/2504.07079.

Huichi Zhou, Siyuan Guo, Anjie Liu, Zhongwei Yu, Ziqin Gong, Bowen Zhao, Zhixun Chen, Menglong Zhang, Yihang Chen, Jinsong Li, Runyu Yang, Qiangbin Liu, Xinlei Yu, Jianmin Zhou, Na Wang, Chunyang Sun, and Jun Wang. Memento-skills: Let agents design agents, 2026. URL https://arxiv.org/abs/2603.18743.

A. Secondary SoPs from Qualitative Analysis

The following SoPs appear in the 122B Deepening +Combined run with moderate support (10–15/323 patches) and are encoded in the evolved skill but not discussed in the main text.

Target-range and answer-position validation (15/323 patches). Before writing, verify the exact target sheet name, cell range, and `answer_position` field from the task metadata. Misreading these fields — writing to the wrong sheet or an off-by-one range — causes silent failures that produce no error message but score zero.

Datatype and datetime preservation (15/323 patches). Write dates and numeric values as native Python types, not strings. Both `pandas` date parsing and `openpyxl` cell assignment can silently stringify datetime values; inspect each column's dtype before writing and use `openpyxl`'s native datetime assignment.

Workbook structure exploration before editing (success-dominant, ~13/323 patches). List all sheets, inspect row/column layout, and verify header positions before any write. This pre-edit exploration prevents wrong-sheet and wrong-range failures and accounts for a substantial share of the 151 success-leaning patches in the run.

B. Prompt Templates and Intermediate Outputs

This appendix reproduces the key prompt templates used in each pipeline stage and illustrates representative intermediate outputs to make the pipeline fully transparent and reproducible.

B.1. Stage 1: Agent System Prompt Template

The agent π_θ operates under the following system prompt during trajectory collection. The skill $\mathcal{S}_0$ is prepended to the user context at inference time. Note that this differs from standard loading process of skills, where initially the agent only has access to skill descriptions. We simplify this by preload the SKILL.md content to system prompt because Trace2Skill focus on improving a fixed target skill which is known relative to the task. Therefore, there is no need for skill selection as the standard skill loading. Importantly, the Trace2Skill skill using agent still need to procedurally discover resources pointed by the preloaded SKILL.md (e.g., resources and scripts), which are not preloaded.

Stage 1 — Agent System Prompt (abbreviated)

Role: You are an expert role (e.g., spreadsheet analysis) agent.
Skill context: `[Contents of` $\mathcal{S}_0$ `inserted here]`
Task: Describing tasks and input files.
Tools available: Describing tools and environment. E.g., `bash` (shell execution) for ReAct with file system access.
Format: ReAct-style interaction — alternate between reasoning traces and tool calls until the task is complete.

B.2. Stage 2: Analyst Prompt Templates and Example Patches

In Stage 2, the patch proposing agents first draw error and success memory items similar to (Ouyang et al., 2026), which are generalizable trajectory-level knowledge that might be helpful for future task executions. Next, the agents read the original skill directory and then propose a patch to encode the memory items into the skill.

B.2.1. Error Analyst Prompt ($\mathcal{A}^-$)

Error Analyst System Prompt (abbreviated)

Role: You are an expert failure-analysis agent for XXX tasks.

Mission: Given an agent's execution artifacts (logs + produced files) and the ground-truth solution, diagnose *why the agent failed*, identify *causal failure reasons*, and *validate* your diagnosis by implementing a minimal fix that makes the agent output match the ground truth. Your analysis must be **systematic**, **evidence-driven**, and **reproducible**. **Do not guess** when you can verify.

Required Workflow (MANDATORY):
1. Understand the task and failure surface — identify exactly what is wrong in the output.
2. Trace the failure to agent behavior — locate the decision or code step that produced the mismatch.
3. Validate the root cause with a minimal fix — write fixed output and re-evaluate against the ground truth.
4. Re-evaluate — if still failing, return to steps 1–3 and revise your diagnosis.

Output: Produce (1) *Failure Cause Items* — systematic, causal reasons grounded in observable agent behavior; (2) *Failure Memory Items* (≤3) — generalizable insights the agent should remember to avoid similar failures.

B.2.2. Success Analyst Prompt ($\mathcal{A}^+$)

Success Analyst System Prompt (abbreviated)

Role: You are an expert in success pattern analysis for AI agent systems.

Mission: Given a successful agent trajectory, identify *generalizable* behavior patterns that contributed to the correct answer.

Requirements:
Broad Coverage — every effective behavior in the trajectory must be captured by a pattern.
Frequency Awareness — patterns covering more instances should be listed first; rare behaviors should be absorbed into the nearest broader pattern.
Generalization — each pattern must describe a general mechanism, not a single task-specific detail.

Output: A compact set of *Success Memory Items* with title, description, and concrete examples of the effective behaviors observed.

B.2.3. Example Trajectory-Level Patch (Error Analyst Output)

Each analyst produces a structured patch for a single trajectory. The following example is from an error analyst ($\mathcal{A}^-$) processing one failure trajectory where the agent deleted rows outside the specified range. The patch proposes adding a new `SKILL.md` section on row-manipulation safety; this patch, along with 52 others citing related structural-edit failures, is later consolidated into the "Structural-edit safety" SoP (§4.4).

Example Stage 2 Trajectory-Level Patch (error analyst, one trajectory)

Analyst reasoning: Single error record reveals a clear failure pattern: the agent deletes rows outside the specified range and fails to distinguish partially-empty data rows from completely empty padding rows. The skill lacks any guidance on row/column manipulation or range boundary enforcement.

```
--- a/SKILL.md
+++ b/SKILL.md
@@ -55,0 +55,26 @@ ## Formula Verification Checklist

+## Row and Column Operations
+
+### Range Boundary Enforcement
+- When a task specifies a cell range (e.g., F14:M83), operations
+  MUST stay within those exact boundaries
+- Parse range format: start_col + start_row : end_col + end_row
+- Never apply operations to rows/columns outside the range
```

```
+
+### Row Deletion Logic
+Before deleting any row, verify BOTH conditions:
+1. Target columns are empty (as specified by task)
+2. Row falls within the explicit deletion range
+
+**Do NOT delete** completely empty rows across ALL columns unless
+explicitly instructed -- these often serve as structural padding.
+
+### Post-Operation Validation
+After row/column operations:
+- [ ] Count remaining rows matches expected count
+- [ ] No header/footer/padding rows outside range were affected
+- [ ] Data integrity is maintained in adjacent columns
+- [ ] Range boundaries were respected (no off-by-one errors)
```

B.3. Stage 3: Merge Operator Prompt and Example Consolidated Patch

B.3.1. Merge Operator Prompt ($\mathcal{M}$)

Merge Operator System Prompt (full)

You are a skill edit coordinator. You receive multiple independently-proposed patches that each suggest changes to a skill folder. Your job is to merge them into one coherent, non-redundant patch.

Guidelines:
1. **Deduplicate**: When multiple patches propose the same or very similar edits, keep the best version (most specific, best worded).
2. **Resolve conflicts**: If patches propose contradictory edits to the same section, choose the one with stronger justification or synthesize both into a better edit.
3. **Preserve unique insights**: Different patches address different failures — include all unique, non-redundant edits.
4. **Maintain conciseness**: The merged patch should have ≤ the sum of unique edits across all input patches. Remove redundancy.
5. **Ensure independence**: Edits in the merged patch MUST be line-level independent — no two edits may target overlapping lines or the same passage of text, even across different operations.
6. **Atomic create/link paairs**: A `create` operation for `references/*.md` and the `SKILL.md` edit that inserts a link to it are an inseparable pair — keep both or drop both.

Prevalent pattern bias: When multiple patches independently propose similar edits addressing the same class of failure or success pattern, treat this recurrence as evidence of a *systematic* property of the task. Preserve such prevalent edits with higher priority and express them as general principles rather than instance-specific fixes.

B.3.2. Example Final Consolidated Patch p^* (After Full Merge Hierarchy)

The following excerpt shows the reasoning and representative edits from the final consolidated patch p^* produced after four levels of hierarchical merging over 323 individual trajectory patches on SpreadsheetBench-Verified.

Example Stage 3 Final Patch Output (excerpt)

```
{
  "reasoning": "Merged 3 patches addressing mixed failure/success
    evidence. Key consolidation decisions: (1) Synthesized recalc.py
    workflow from all patches using the most prominent CRITICAL WARNING
    placement, CSV fallback validation, and a verification loop;
    (2) Consolidated library selection guidance into a comprehensive
    decision tree; (3) Combined row deletion guidance emphasizing
    bottom-up/right-to-left deletion order from all patches;
    (4) Merged formula validation checklists without redundancy.",
  "edits": [
```

```
    {
      "file": "SKILL.md",
      "op": "insert_after",
      "target_section": "# Requirements for Outputs",
      "content": "## Important Automation Guidelines\n\n
        **Prefer Python over VBA for Automation**: When tasks request
        VBA macros or spreadsheet automation, implement the logic in
        Python using openpyxl/pandas instead. This provides better
        error handling, easier debugging, cross-platform compatibility,
        and avoids macro security issues."
    },
    {
      "file": "SKILL.md",
      "op": "insert_after",
      "target_section": "## Important Requirements",
      "content": "### When to Compute Directly in Python\n
        If LibreOffice/recalc.py cannot evaluate certain formula
        patterns (complex array formulas, advanced INDEX/MATCH
        combinations), compute the logic directly in Python and write
        results as values. Use this fallback when formula returns errors
        after recalculation despite correct syntax."
    }
  ],
  "changelog_entries": [
    "Added automation guidelines preferring Python over VBA",
    "Consolidated recalc.py warnings into a single CRITICAL WARNING"
  ]
}
```


B.3.3. Example Translated Diff: Final Consolidated Patch p^*

The JSON patch p^* produced by Stage 3 is translated programmatically into a unified diff and applied directly to the skill directory. The following excerpt shows two hunks from the final applied diff on SpreadsheetBench-Verified, corresponding to the two most prevalent SoPs: formula recalculation enforcement (178/323 patches) and tool-selection guidance (177/323 patches).

Example Stage 3 Applied Diff (excerpt from final p^*)

```
--- a/SKILL.md
+++ b/SKILL.md
@@ -126,3 +261,18 @@ ## Common Workflow

 ## Common Workflow

+### CRITICAL WARNING: Formula Recalculation Is Mandatory
+
+**If you write ANY formulas to an Excel file using openpyxl,
+you MUST run recalc.py before considering the task complete.**
+
+Formulas written via openpyxl exist only as text strings until
+recalculated. Without running recalc.py:
+- Cells return None/empty when read with data_only=True
+- Evaluation fails even if formulas are syntactically correct
+- The output file is incomplete
+
+This is non-negotiable. Do not proceed to verification or
+delivery until recalc.py confirms success.
+
@@ -138,3 +285,9 @@ ### Working with openpyxl
```

```
+### Tool Selection Warning
+
+**CRITICAL**: When modifying spreadsheets that contain existing
+formulas you need to preserve:
+- Use openpyxl (load_workbook() then save()) -- formulas remain
+  as strings
+- Avoid pandas (to_excel()) -- silently converts formulas to
+  static values permanently
```